Morning Cheat Sheet — read this and nothing else

GCC Java Backend · NCG · Monday 16 November · Learn nothing new today — revision only.

45 minutes. Nothing else. No new material.
If a line here does not ring a bell, open that file for three minutes and come back.

The 8 answers that carry this interview

1. **HashMap:** *"`hashCode()`, then a spread of `h ^ (h >>> 16)`, then bucket index `(n-1) & hash`. Collisions chain in a linked list, and since Java 8 a bucket over 8 entries becomes a red-black tree. Default capacity 16, load factor 0.75, so it resizes and rehashes at 12."*
2. **equals/hashCode:** *"Equal objects must have equal hash codes. Override `equals` without `hashCode` and the map looks in the wrong bucket and never finds the entry."*
3. **N+1:** *"One query for the list, then one per row for the relation. Fix with JOIN FETCH, `@EntityGraph`, batch fetching, or a DTO projection. That is what took my report from 15 seconds to 6."*
4. **`@Transactional` internal call:** *"It works through a proxy. An internal `this.method()` call bypasses the proxy, so no transaction starts."*
5. **Constructor injection:** *"Fields can be final, dependencies cannot be missing, and I can test with `new Service(mock)` without a Spring context."*
6. **Kafka vs RabbitMQ:** *"RabbitMQ is a postbox — delivered to a worker, then gone. Kafka is a ledger — events are retained and many consumer groups read independently. I used RabbitMQ for email tasks and Kafka for state-change events with multiple consumers."*
7. **Circuit breaker:** *"Closed, open, half-open. Failing fast stops the caller exhausting its threads and gives the failing service room to recover."*
8. **Index cost:** *"Faster reads, but every insert and update maintains every index, so I add them deliberately, not by default."*

Containers and CI/CD ●● — your biggest gap, named 3× in the JD

Say the boundary first: *"I have not owned the container and deployment side — that was a separate team. So, understanding rather than hands-on."* **Then explain it properly.**

- **Container vs VM** — a VM carries a whole guest OS; a container shares the host kernel and packages only the app and its dependencies. MBs and seconds, not GBs and minutes. VMs isolate more strongly.
- **Dockerfile → image → container** = recipe → cake → slice. **One image, many containers.**
- **Layer caching** — copy `pom.xml` and download dependencies *before* copying `src`, so a code change does not re-download everything. **Multi-stage build** throws away Maven and the JDK so the shipped image carries only the JRE and the JAR.
- **Config comes from env vars**, never baked into the image — same artefact everywhere, only configuration changes. Never put secrets in an image.
- **Pod** disposable, new IP each time · **Deployment** gives rolling updates and rollback · **Service** = a stable name and load balancing in front of changing pods — *"the role Eureka played in my Classifieds project"*.
- ● **Readiness vs liveness** — ready for traffic yet, versus still alive. Confusing them is a real incident: a slow Spring Boot startup failing a *liveness* probe gets killed and restarted forever.
- **Rolling updates mean two versions run at once** → database migrations must be backwards compatible. Add a nullable column, deploy, then use it. **Expand then contract.**
- **CrashLoopBackOff** → `kubectl logs --previous`; usually a missing env var or a DB it cannot reach. **ImagePullBackOff** → wrong tag or registry credentials.
- **CI/CD:** build → test → quality gate → package → deploy. ● **Build once, promote the same artefact** through SIT, UAT, prod — rebuilding per environment means you test one thing and ship another. Jenkins / GitHub Actions / Azure DevOps all express this same shape.
- **In BFSI the production step is gated and approved**, not automatic. Say that.

OAuth2 vs JWT ●

*"Not alternatives. **OAuth2 is how a token is obtained; JWT is a format a token can be written in.**"* Four roles: resource owner, client, authorization server, resource server. **Authorization Code flow** — the app never sees the password; the code is exchanged server to server so the token never sits in a URL. **PKCE** for mobile and SPAs. **Client credentials** for service-to-service. **OIDC is an identity layer on top of OAuth2** — OAuth2 authorises, OIDC authenticates. *Your honest line: you issued JWTs from your own service rather than integrating a full authorization server.*

ActiveMQ = a JMS broker, conceptually RabbitMQ not Kafka — queue or topic, delivered then gone. Still common in older banking estates, which is why it is on the JD.

Core Java

- **Overloading** = compile time, different parameters. **Overriding** = runtime, same signature, subclass. You cannot override a static method — it is hidden.
- **Abstract class** = shared state and code, one only. **Interface** = capability contract, many.

- **ArrayList** = array, O(1) get. **LinkedList** = nodes, O(n) get. Default to ArrayList.
- **HashMap** not thread safe · **Hashtable** locks the whole map, legacy · **ConcurrentHashMap** locks per bucket.
- **String immutable** because of the string pool, security, thread safety, cached hash.
- `==` compares references, `.equals()` compares values. `new String("a") != "a"`.
- **Checked** = compiler forces handling. **Unchecked** extends `RuntimeException`.
- `finally` always runs except on `System.exit()`. Never `return` from `finally`.
- **Fail-fast** throws `ConcurrentModificationException`; remove via `iterator.remove()`.

Java 8

- **Intermediate** ops are lazy (`filter`, `map`, `sorted`); **terminal** ops trigger (`collect`, `reduce`, `forEach`, `findFirst`).
- `map` = one to one. `flatMap` = one to many, then flattened.
- `Predicate`→filter · `Function`→map · `Consumer`→forEach · `Supplier`→lazy value.
- `groupingBy(X, counting())` and `groupingBy(X, summingDouble(Y))`.
- **Optional:** use `map` / `orElse` / `orElseThrow`, not `isPresent` + `get`.
  `orElse` always evaluates; `orElseGet` is lazy.

Spring Boot

- **IoC** = the framework controls creation. **DI** = how it hands dependencies in.
- Beans are **singleton by default**, so keep them **stateless** — a mutable instance field is a concurrency bug.
- `@Repository` also **translates database exceptions** into Spring's hierarchy.
- `@RestControllerAdvice` + `@ExceptionHandler` = one consistent error shape. Log the stack trace, return a generic message.
- **Auto-configuration** works through `@ConditionalOnClass` and `@ConditionalOnMissingBean` — your own bean always wins.
- `@WebMvcTest` = slice, fast · `@SpringBootTest` = full context · `@MockBean` = replace a bean.

JPA and Hibernate

- **JPA** = spec · **Hibernate** = implementation · **Spring Data JPA** = the layer on top.
- Defaults: `@ManyToOne` and `@OneToOne` are **EAGER**; `@OneToMany` and `@ManyToMany` are **LAZY**. Set ManyToOne to LAZY explicitly.
- `@Transactional` rolls back on **unchecked** exceptions only, unless `rollbackFor`.
- **Dirty checking** means a managed entity needs no `save()`.
- First-level cache is always on, per transaction. Second-level is opt-in and shared.
- Use `@Enumerated(EnumType.STRING)`, never the ORDINAL default.
- Money is **BigDecimal**, never `double`.

Microservices • *(mandatory in Posting B — and all over your resume)*

- Gains: independent deploy, independent scale, fault isolation. **Costs: network failure, no distributed transaction, harder debugging.** Say the costs.
- **Eureka** = discovery · **API Gateway** = single entry, auth and routing in one place · **Config Server** = externalised config.
- **Saga** = local transactions plus **compensating** transactions. Eventual consistency.
- Only retry **idempotent** operations. Always set a **timeout**.
- Split by **business capability**, not technical layer.
- Observability: centralised logs, **correlation ID** tracing, metrics.

Messaging

- Kafka: **topic → partitions → offsets**; order is guaranteed **within a partition**, so key by transaction or account ID.
- **Same consumer group** = share the partitions. **Different groups** = each gets everything.
- **At-least-once** is the normal default, so consumers must be **idempotent**.
- RabbitMQ: producer → **exchange** → queue. Direct, topic, fanout. **DLQ** after N failed attempts.

Security

- **JWT** = header.payload.signature. **Signed, not encrypted** — anyone can read the payload, so put nothing sensitive in it.
- Stateless, so any instance validates it — that is why it suits microservices.
- Cannot easily revoke: short-lived access token plus refresh token, or a blocklist.
- CSRF is off for a stateless API because the browser does not auto-attach an `Authorization` header.
- **BCrypt** for passwords — slow and salted by design.
- `hasRole("ADMIN")` looks for the authority `ROLE_ADMIN`.

SQL

- **Left-most prefix rule:** an index on `(status, created_at)` does not serve `created_at` alone.
- Index killers: a **function on the column**, and a **leading** `%` in LIKE.
- `EXPLAIN`: `type: ALL` means full scan · check `key`, `rows`, and `Extra`.
- `WHERE` filters rows **before** grouping; `HAVING` filters groups **after**.
- `UNION` deduplicates and sorts; `UNION ALL` does not and is faster.

- Customers with no transactions = `LEFT JOIN ... WHERE t.id IS NULL`.
- Deep `OFFSET` is slow — prefer **keyset pagination**.

Coding round

- Talk while you type. Ask about edge cases **first**. State the complexity at the end.
- `LinkedHashMap` for first-non-repeating. `seen.add(n)` returns false for a duplicate.
- Two sum = HashMap, O(n), trading space for time. Mention the brute force first.
- `Integer` cache is −128 to 127, so `128 == 128` is **false** for boxed Integers.

Delivery half of the JD • — most candidates prepare none of this

- **Code review, what you look for, in order:** correctness and edge cases · error handling (nothing swallowed silently) · **the data layer** — N+1 in a loop, missing index, `@Transactional` on the right method · meaningful tests · readability last.
- **Giving feedback:** separate must-change from suggestion and say which · ask rather than instruct · past three replies, go and talk to them.
- **Unit test plan review:** cover **behaviour and rules**, not lines. The valuable tests are the **negative** ones — an approved request cannot be approved twice, a requester cannot approve their own. *(You wrote exactly these.)*
- **Coverage:** tells you what ran, not what was verified. A test with no assertion still counts.
- **Estimation:** break it down until you can picture doing each piece · include testing, review turnaround and **integration** · **state the assumptions**, so a change is a re-estimate rather than a slip.
- **Technical risk planning:** what could go wrong, how likely, how bad, what we do about it.
  Your example: moving email to RabbitMQ risked silent failure, so retries with backoff, a DLQ, and alerting on the DLQ. Async without a DLQ moves the failure somewhere nobody is looking.
- **Design review:** does it meet the requirement · what happens when a dependency is down · single points of failure · authorisation server side · how would we know it works in production · how do we roll back · **is it more complex than the problem needs**.
- **Client raises a quality issue:** acknowledge fast · contain first, cause second · do not commit to a cause before you know, do commit to when you will update · fix, verify, say what changed so it does not recur. *"Being wrong quickly and openly costs far less than being confident and wrong slowly."*
- **Leading a team:** be honest — you have not led one. You have reviewed code and paired.
  Explaining *why* sticks; pointing at the answer does not.
- **Knowledge management:** Swagger docs that live with the code · runbooks · handover notes.
  Test = could someone pick up my work if I were away.
- **High quality code:** correct · readable · tested where it matters · performs at real data volumes (both your fixes were fine in testing and slow in production) · **traceable** — logging and audit trail, which in a bank is part of the deliverable.

Your stories — the numbers

15s → 6s report: N+1 found via SQL logging, fixed with a projection query, indexes, rewritten JOINs. *"My first guess was wrong, which is why I measure first."*
2–3s → under 1s APIs: N+1 plus lazy loading plus indexing.
RabbitMQ: email off the request thread, bulk approvals stopped timing out.
Kafka: state changes to multiple consumer groups, retained and replayable.
RBAC: requester, approver, admin — enforced server side, because a UI check only hides the button.
Classifieds: 3 services, own databases, Eureka, API Gateway, JWT, global exception handling.

The three questions you must not fumble

"Tell me about yourself" → Now → What you built → Why this role. Under 2 minutes.

"Walk me through your timeline." → B.Tech 2022 · JSPIDERS Java programme May 2023 – Jan 2024 · Artifint technical recruitment Feb – Dec 2024 · Zensar from Dec 2024. The one gap is 2022 to mid-2023 — two sentences, the **real** reason, no apology.

"Why recruitment, then development?" → B.Tech in CS, development was always the plan, JSPIDERS to build the skills, and the Artifint role was **technical** recruitment so I was close to the work. **Then turn it:** *"That year is why I am comfortable in client and requirement conversations"* — which is the first responsibility listed in both postings.

The three answers to rehearse most *(more than any technical one)*:

Experience band (JD says 3–6) → *"In engineering, December 2024 to June 2026 — about a year and a half — and around two and a half years professional overall. I know that is below the range."* Then straight to evidence: UK bank platforms, **15s → 6s**, **2–3s → under 1s**, Kafka and RabbitMQ, production incidents. **Never round up — they verify against payslips.**

The gap (you left in June) → two sentences, the real reason, then **specifically** what you did since. *"I have been working through containerisation and deployment, which was the part of my last role I was furthest from"* is honest and it is exactly what this JD asks for. Close on being available immediately.

Recruitment → development → B.Tech CS, JSPIDERS, and Artifint was **technical** recruitment.
Then turn it: *"that year is why I am comfortable in client and requirement conversations"* — and this is a **client-facing GCC role**.

"How much experience do you have?" → Answer with dates, never a rounded number.

"Engineering since December 2024, so about a year and nine months, plus a year in technical recruitment before that."

Questions to ask them "Which domain would this role sit in?" · "Greenfield microservices or modernising a monolith?" · "What would a successful first three months look like?" ·

"What is the biggest technical problem this hire would help with?"

The last five minutes

- You have real production numbers and banking domain experience. Most candidates for this role have neither.
- **If you do not know something: say so, then reason out loud.** That has never cost anyone an offer. Bluffing has.
- Give the **why**, not just the **what**. "HashMap stores key-value pairs" is a textbook answer; "O(1) because it hashes to a bucket instead of scanning" is an engineer's.
- Sit up. Breathe out slowly. Smile before you speak — it changes your voice.

Go get it.